

Exam 15-11-2024 - Selected Solutions

Problem 2: Server and Infrastructure Activation (Max-Flow Min-Cut)

1. Algorithm Design (Graph Construction)

Il problema di decidere quali server e infrastrutture attivare per massimizzare il profitto [cite: 7] può essere modellato in modo ottimo come un problema di Max-Weight Closure, risolvibile tramite Max-Flow Min-Cut.

Costruiamo un grafo orientato $G = (V, E)$ nel seguente modo: * **Nodi**: Definiamo un nodo sorgente S , un nodo pozzo T , un nodo per ogni server $i \in [n]$ e un nodo per ogni infrastruttura $j \in [m]$ [cite: 7]. * **Archi dei Ricavi**: Aggiungiamo un arco dalla sorgente S a ogni server i con capacità pari al ricavo, ovvero $c(S, i) = r(i)$ [cite: 7]. * **Archi dei Costi**: Aggiungiamo un arco da ogni infrastruttura j al pozzo T con capacità pari al costo, ovvero $c(j, T) = c(j)$ [cite: 7]. * **Archi di Dipendenza**: Poiché il server i richiede l'attivazione dell'infrastruttura $j(i)$, aggiungiamo un arco dal server i all'infrastruttura $j(i)$ con capacità infinita, $c(i, j(i)) = \infty$ [cite: 7].

Ricerca della Soluzione Ottima: Calcoliamo il flusso massimo da S a T . Troviamo il taglio minimo (A, B) identificando tutti i nodi raggiungibili da S nel grafo residuo (che formeranno l'insieme A). I server e le infrastrutture da attivare per massimizzare il profitto sono esattamente i nodi che appartengono all'insieme A .

2. Proof of Correctness

Dimostriamo che minimizzare la capacità del taglio (A, B) equivale a massimizzare il profitto aziendale [cite: 7].

In un taglio minimo di capacità finita, nessun arco con capacità ∞ può attraversare il taglio da A a B . Questo garantisce l'ammissibilità della soluzione: se un server i viene inserito in A (attivato), la sua infrastruttura $j(i)$ non può trovarsi in B , ma deve necessariamente trovarsi anch'essa in A (attivata) [cite: 7].

La capacità del taglio è la somma delle capacità degli archi che vanno da A a B : * Archi (S, i) per i server $i \in B$ (server non attivati). * Archi (j, T) per le infrastrutture $j \in A$ (infrastrutture attivate).

$$c(A, B) = \sum_{i \in B} r(i) + \sum_{j \in A} c(j)$$

Sia $R = \sum_{i \in [n]} r(i)$ il ricavo totale che si otterrebbe attivando tutti i server [cite: 7]. Possiamo riscrivere R suddividendolo sui due insiemi della partizione: $R = \sum_{i \in A} r(i) + \sum_{i \in B} r(i)$, da cui ricaviamo $\sum_{i \in B} r(i) = R - \sum_{i \in A} r(i)$. Sostituendo questa espressione nell'equazione del taglio:

$$c(A, B) = \left(R - \sum_{i \in A} r(i) \right) + \sum_{j \in A} c(j)$$

$$c(A, B) = R - \left(\sum_{i \in A} r(i) - \sum_{j \in A} c(j) \right)$$

Il termine tra parentesi è esattamente il **Profitto** totale (ricavi meno costi) della nostra selezione in A [cite: 7]. Avendo stabilito che $c(A, B) = R - \text{Profitto}$, e sapendo che R è una costante dell'istanza, è matematicamente dimostrato che l'algoritmo di Min-Cut, trovando il taglio (A, B) di valore minimo, estrae necessariamente la selezione che massimizza il Profitto.

3. Computational Complexity

Definiamo la dimensione della rete creata in funzione dell'input fornito [cite: 7]: * **Numero di vertici** ($|V|$): n server, m infrastrutture, più sorgente e pozzo. Quindi

$|V| = n + m + 2 = O(n + m)$ [cite: 7]. * **Numero di archi** ($|E|$): n archi in uscita da S , m archi in ingresso a T , e n archi di dipendenza. Quindi $|E| = 2n + m = O(n + m)$ [cite: 7].

Per il calcolo del flusso massimo possiamo applicare un algoritmo standard fortemente polinomiale come **Edmonds-Karp** [cite: 7]. La complessità canonica di Edmonds-Karp è $O(|V| \cdot |E|^2)$.

Sostituendo le variabili calcolate per il nostro modello, otteniamo:

$$O((n + m) \cdot (n + m)^2) = O((n + m)^3)$$

L'algoritmo trova quindi la soluzione ottima in tempo polinomiale $O((n + m)^3)$ [cite: 7].

Problem 3: Critical Edges and Capacity Reduction

1. Proof: Any Edge Crossing a Minimum Cut is Critical

Sia $G = (V, E)$ una rete di flusso con capacità $c(e)$, sorgente s , pozzo t e valore di flusso massimo $F > 0$ [cite: 6]. Un arco e è definito *critico* se una diminuzione della sua capacità comporta una riduzione del valore del flusso massimo [cite: 6].

Dimostrazione: 1. Per il teorema del Flusso Massimo e Taglio Minimo (Max-Flow Min-Cut Theorem), esiste un taglio s - t partizionato in (S, T) (con $s \in S$ e $t \in T$) tale che la sua capacità sia pari al valore del flusso massimo iniziale:

$$\text{cap}(S, T) = \sum_{e \in \delta^+(S)} c(e) = F$$

dove $\delta^+(S) = (u, v) \in E \mid u \in S, v \in T$ è l'insieme degli archi orientati che attraversano il taglio da S verso T . 2. Sia $\hat{e} \in \delta^+(S)$ un qualsiasi arco che attraversa questo taglio minimo da S verso T . Riduciamo la capacità di $\hat{e}$ di una quantità strettamente positiva $\epsilon > 0$ (o di 1 unità discreta), definendo la nuova funzione di capacità c' :

$$c'(\hat{e}) = c(\hat{e}) - \epsilon$$

$$c'(e) = c(e) \quad \forall e \neq \hat{e}$$

3. Calcoliamo la capacità del medesimo taglio (S, T) con le nuove capacità c' :

$$\text{cap}'(S, T) = \sum_{e \in \delta^+(S)} c'(e) = (c(\hat{e}) - \epsilon) + \sum_{e \in \delta^+(S) \setminus \hat{e}} c(e) = \text{cap}(S, T) - \epsilon = F - \epsilon$$

4. Per la proprietà di dualità debole (*Weak Duality*), il nuovo flusso massimo F' nella rete con capacità ridotte non può superare la capacità di alcun taglio s - t . Di conseguenza:

$$F' \leq \text{cap}'(S, T) = F - \epsilon < F$$

Poiché il flusso massimo F' diminuisce strettamente a seguito della riduzione della capacità di $\hat{e}$, l'arco $\hat{e}$ è critico [cite: 6].

2. Algorithm to Decrease Max-Flow with $c(e) \in 0, 1$ by Deleting at Most k Edges

Assumiamo che ogni arco abbia capacità unitaria $c(e) \in 0, 1$ e sia $k \leq F$ [cite: 6]. Eliminare un arco con $c(e) = 1$ equivale ad azzerarne la capacità, riducendo di 1 la capacità di qualsiasi taglio che attraversa.

Strategia: Poiché $F = \text{cap}(S, T)$ in un taglio minimo (S, T) ed ogni arco ha capacità 1 [cite: 6], il taglio minimo $\delta^+(S)$ è composto esattamente da F archi disgiunti [cite: 6]. Eliminare un arco da un taglio minimo riduce la capacità di quel taglio di esattamente 1. Se eliminiamo k archi qualsiasi appartenenti a $\delta^+(S)$, la nuova capacità del taglio (S, T) diventerà:

$$\text{cap}'(S, T) = F - k$$

Poiché il valore del nuovo flusso non può superare la capacità di alcun taglio, il flusso massimo scenderà al più a $F - k$. Poiché ogni eliminazione di arco può ridurre il flusso di al massimo 1 unità (essendo le capacità unitarie), una riduzione di k unità è la massima teoricamente raggiungibile rimuovendo k archi.

Algoritmo: 1. Calcola il flusso massimo f su G (ad esempio con Ford-Fulkerson o BFS di Edmonds-Karp). 2. Nel grafo residuo finale G_f , esegui una visita (BFS/DFS) partendo da s . Sia S l'insieme dei nodi raggiungibili da s , e $T = V \setminus S$. 3. Identifica gli archi del taglio minimo: $E_{cut} = (u, v) \in E \mid u \in S, v \in T$. L'insieme E_{cut} conterrà esattamente F archi con capacità 1 [cite: 6]. 4. Seleziona un qualsiasi sottoinsieme di k archi da E_{cut} e rimuovili dal grafo.

3. Time Complexity

- Con capacità binarie $c(e) \in \{0, 1\}$, il flusso massimo è intero e limitato da $|V|$ (o $|E|$) [cite: 6]. L'algoritmo di Ford-Fulkerson termina in tempo $O(|E| \cdot F)$ [cite: 6].
 - La BFS/DFS nel grafo residuo per trovare l'insieme S richiede tempo $O(|V| + |E|)$.
 - L'identificazione degli archi in E_{cut} e la rimozione di k di essi richiede tempo $O(|E|)$.
 - La complessità computazionale totale è determinata dal calcolo del flusso: $O(|E| \cdot F)$ [cite: 6].
-

Problem 4: Homework Late Penalties (Scheduling with Deadlines)

1. Optimal Greedy Algorithm

Ogni compito i richiede d_i giorni e comporta una penalità di c_i punti al giorno [cite: 6]. Il tempo di completamento t_i accumula i giorni necessari per completare l'attività i e tutte quelle che la precedono nella schedulazione [cite: 6].

Criterio Greedy (Smith's Rule): Ordiniamo i compiti in ordine decrescente rispetto al rapporto tra penalità giornaliera e tempo di esecuzione:

$$\frac{c_i}{d_i}$$

In caso di parità nel rapporto, l'ordine tra compiti equivalenti può essere scelto arbitrariamente.

Pseudocodice:

```
Algorithm MinimizeLatePenalty(A, n):
  Compute ratio  $r_i = c_i / d_i$  for all  $i$  in  $\{1, \dots, n\}$ 
  Sort the assignments in descending order of  $r_i$ 
  Return the sorted sequence of assignments
```

2. Proof of Correctness (Exchange Argument)

Dimostriamo che la schedulazione prodotta dalla regola greedy è ottima tramite un argomento di scambio (*Exchange Argument*).

Supponiamo per assurdo che esista una schedulazione ottima O diversa dall'ordinamento greedy e che presenti il minor numero possibile di inversioni rispetto ad esso. Poiché O non rispetta l'ordine decrescente dei rapporti, devono esistere due compiti adiacenti i e j , con i eseguito immediatamente prima di j , tali che:

$$\frac{c_i}{d_i} < \frac{c_j}{d_j} \iff c_i \cdot d_j < c_j \cdot d_i$$

Sia T il tempo totale impiegato per completare tutti i compiti che precedono la coppia (i, j) nella schedulazione O . Nella schedulazione O : * Il compito i termina al giorno: $t_i = T + d_i$ * Il compito j termina al giorno: $t_j = T + d_i + d_j$

Il contributo della coppia (i, j) alla penalità totale in O è:

$$\text{Penalty}(i, j) = c_i(T + d_i) + c_j(T + d_i + d_j)$$

Costruiamo una nuova schedulazione O' scambiando l'ordine di esecuzione dei soli compiti i e j (eseguendo prima j e poi i). Tutti i compiti precedenti a j e tutti i successivi a i completano esattamente negli stessi giorni, poiché la somma totale dei giorni $d_j + d_i = d_i + d_j$ non varia. Nella nuova schedulazione O' : * Il compito j termina al giorno: $t'_j = T + d_j$ * Il compito i termina al giorno: $t'_i = T + d_j + d_i$

Il contributo della coppia (j, i) alla penalità totale in O' è:

$$\text{Penalty}'(j, i) = c_j(T + d_j) + c_i(T + d_j + d_i)$$

Calcoliamo la differenza tra le due penalità:

$$\text{Penalty}(i, j) - \text{Penalty}'(j, i) = [c_i(T + d_i) + c_j(T + d_i + d_j)] - [c_j(T + d_j) + c_i(T + d_j + d_i)]$$

Sviluppando i prodotti ed eliminando i termini comuni ($c_i T$, $c_j T$, $c_i d_i$, $c_j d_j$):

$$\text{Penalty}(i, j) - \text{Penalty}'(j, i) = c_j \cdot d_i - c_i \cdot d_j$$

Poiché per ipotesi avevamo $\frac{c_i}{d_i} < \frac{c_j}{d_j}$, ne consegue che $c_j \cdot d_i - c_i \cdot d_j > 0$. Di conseguenza:

$$\text{Penalty}'(j, i) < \text{Penalty}(i, j)$$

Lo scambio ha prodotto una schedulazione con penalità strettamente inferiore rispetto a O , contraddicendo l'ipotesi che O fosse ottima. Dunque l'ordinamento greedy è ottimo.

3. Example

Consideriamo i dati dell'esempio fornito nel testo d'esame [cite: 6]: * $A_1: d_1 = 2$, $c_1 = 10 \implies \frac{c_1}{d_1} = \frac{10}{2} = 5$ [cite: 6] * $A_2: d_2 = 3$, $c_2 = 30 \implies \frac{c_2}{d_2} = \frac{30}{3} = 10$ [cite: 6] * $A_3: d_3 = 2$, $c_3 = 5 \implies \frac{c_3}{d_3} = \frac{5}{2} = 2.5$ [cite: 6]

Ordinando per rapporti decrescenti ($\frac{c_i}{d_i}$):

$$10 > 5 > 2.5 \implies \text{Ordine: } (A_2, A_1, A_3)$$

[cite: 6]

Calcolo dei tempi di completamento e penalità: * A_2 termina al giorno

$$t_2 = 3 \implies \text{costo} = 3 \cdot 30 = 90 \text{ [cite: 6]} * A_1 \text{ termina al giorno}$$

$$t_1 = 3 + 2 = 5 \implies \text{costo} = 5 \cdot 10 = 50 \text{ [cite: 6]} * A_3 \text{ termina al giorno}$$

$$t_3 = 3 + 2 + 2 = 7 \implies \text{costo} = 7 \cdot 5 = 35 \text{ [cite: 6]} * \textbf{Penalità Totale: } 90 + 50 + 35 = 175 \text{ [cite: 6]}.$$

4. Computational Complexity

- Il calcolo dei rapporti $r_i = c_i/d_i$ per tutti gli n compiti richiede tempo lineare $O(n)$.
- L'ordinamento degli n elementi in base al rapporto richiede tempo $O(n \log n)$ utilizzando un algoritmo basato su confronti come MergeSort o QuickSort.
- Tempo totale di esecuzione: $O(n \log n)$.
- Complessità spaziale: $O(n)$ per memorizzare l'array delle attività e dei relativi indici/rapporti.